Entrega 2 – A3 – Gestão e Qualidade de Software

Guilherme Araujo Mesquita - 12524228428 - guilherme.0603@hotmail.com

Davi Bessa Araujo – 12526126351 -bessadavi4@gmail.com

Giovanna de Souza Húngaro – 1252429573 - giovanna-hungaro@hotmail.com

Marco Antônio da Silva Viana – 12524229714 - marcowatjob@gmail.com

Giovanna Gomes Leal – 12524236666 - giovannag197@gmail.com

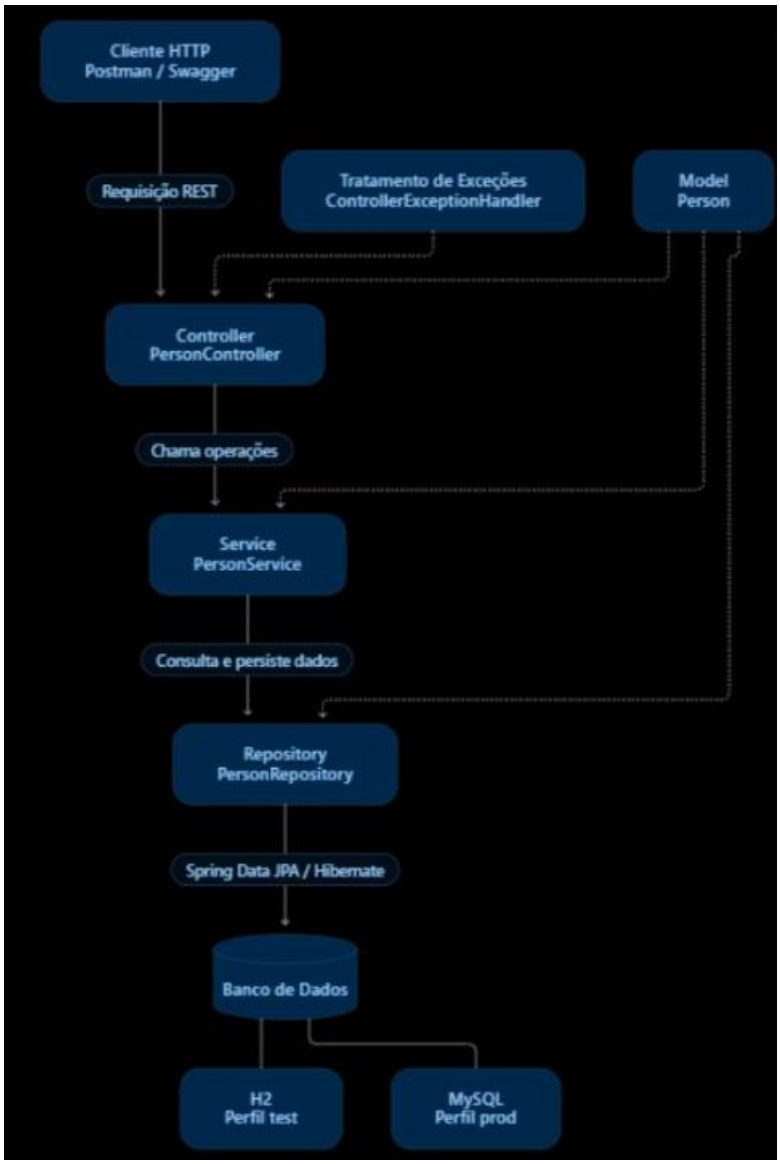
DOCUMENTAÇÃO TÉCNICA – PROJETO REST SPRING TEST

1. Documentação da Arquitetura

1.1. Visão geral do sistema

O projeto consiste em uma API REST desenvolvida em Java com Spring Boot, voltada ao gerenciamento de dados da entidade Person (Pessoa).

A aplicação foi organizada em camadas, separando as responsabilidades de recebimento das requisições, processamento das operações e acesso ao banco de dados. Essa divisão facilita a manutenção do código e permite testar cada parte do sistema de forma independente.



1.2. Estrutura da arquitetura

A arquitetura é composta pelos seguintes elementos:

Cliente HTTP (Postman/Swagger): responsável por enviar requisições à API e receber as respostas. É por meio dessas ferramentas que são realizadas as verificações das funcionalidades.

PersonController: recebe as requisições HTTP e direciona as operações para a camada de serviço. É a camada de comunicação entre o usuário e a aplicação.

PersonService: concentra o processamento das operações relacionadas à entidade Person, fazendo a ligação entre o controller e o repositório.

PersonRepository: responsável pelo acesso aos dados. Utiliza os recursos do Spring Data JPA para realizar operações de consulta e persistência.

Model Person: representa a entidade utilizada pela aplicação e seus dados.

ControllerExceptionHandler: responsável pelo tratamento centralizado das exceções, permitindo que erros ocorridos durante as requisições sejam tratados de maneira padronizada.

Spring Data JPA/Hibernate: realiza a comunicação entre a aplicação Java e o banco de dados relacional.

1.3. Fluxo de funcionamento

O funcionamento da aplicação acontece da seguinte maneira:

1. O cliente envia uma requisição HTTP por meio do Postman ou Swagger.
2. O PersonController recebe a solicitação.
3. O controller encaminha a operação para o PersonService.
4. O serviço realiza o processamento necessário e solicita acesso aos dados ao PersonRepository.
5. O repositório utiliza o Spring Data JPA/Hibernate para consultar ou persistir informações.
6. O banco de dados retorna o resultado da operação.
7. A resposta retorna pelas camadas até chegar ao cliente.

Quando ocorre uma exceção, o ControllerExceptionHandler realiza o tratamento correspondente.

1.4. Tecnologias utilizadas

- Java 21: linguagem utilizada no desenvolvimento.
- Spring Boot: framework responsável pela estrutura da aplicação.
- Spring Data JPA: facilita o acesso e a persistência dos dados.
- Hibernate: implementação utilizada para o mapeamento objeto-relacional.
- H2: banco de dados utilizado no perfil de testes.
- MySQL: banco de dados utilizado no perfil de produção.
- Maven: gerenciamento de dependências e execução do projeto.
- Postman e Swagger: ferramentas de comunicação e documentação da API.
- JUnit 5 e Mockito: ferramentas para testes automatizados.
- TestContainers e Rest-Assured: apoio aos testes de integração e de API.

- JaCoCo e Maven Surefire: geração de relatórios de cobertura e execução dos testes.

1.5. Configuração dos ambientes

O projeto trabalha com dois perfis principais:

Perfil de testes (test): utiliza o banco H2, permitindo executar os testes sem depender de uma instalação externa do MySQL.

Perfil de produção (prod): utiliza o MySQL para armazenar os dados da aplicação em um banco relacional persistente.

Essa separação permite testar as funcionalidades em um ambiente isolado antes de utilizar o banco configurado para produção.

2. PLANO DE TESTES

2.1. Objetivo

O objetivo dos testes é verificar se as funcionalidades da API estão funcionando corretamente, se as camadas se comunicam como esperado e se os dados são armazenados e recuperados sem inconsistências.

Os testes também buscam identificar possíveis falhas antes da utilização da aplicação em um ambiente de produção.

2.2. Tipos de testes

Testes unitários

Serão utilizados para verificar o comportamento individual das classes, principalmente PersonService e suas operações. O Mockito poderá simular o comportamento do repositório, permitindo testar a lógica do serviço sem depender diretamente do banco de dados.

Ferramentas: JUnit 5 e Mockito.

Testes da camada de repositório

Serão utilizados para verificar se as operações de persistência e consulta funcionam corretamente, incluindo a comunicação com o banco de dados.

Ferramentas: JUnit 5, Spring Boot Test e H2.

Testes da camada de controller

Serão utilizados para verificar o recebimento das requisições HTTP, o encaminhamento das operações e o retorno das respostas ao cliente.

Ferramentas: JUnit 5, Mockito e recursos de teste do Spring.

Testes de integração

Serão utilizados para verificar a comunicação entre diferentes partes da aplicação, principalmente entre os serviços, repositórios e banco de dados.

Ferramentas: TestContainers e Rest-Assured.

Testes manuais da API

Serão realizados utilizando Postman ou Swagger, enviando requisições e conferindo os códigos HTTP e os dados retornados.

2.3. Cenários de testes planejados

ID	Cenário	Resultado esperado
CT01	Consultar os registros de pessoas	A API retorna os registros disponíveis.
CT02	Cadastrar uma pessoa com dados válidos	A pessoa é registrada corretamente.
CT03	Consultar uma pessoa existente	Os dados correspondentes são retornados.
CT04	Consultar uma pessoa inexistente	A aplicação trata a situação adequadamente.
CT05	Atualizar os dados de uma pessoa	As alterações são persistidas corretamente.
CT06	Excluir uma pessoa	O registro é removido conforme a operação prevista.
CT07	Enviar dados inválidos	A aplicação trata a entrada inválida.
CT08	Simular uma falha durante uma operação	A exceção é tratada pelo mecanismo correspondente.
CT09	Verificar a persistência no banco	Os dados são armazenados e recuperados corretamente.
CT10	Executar a aplicação no perfil de testes	A aplicação utiliza o banco H2.
CT11	Executar a aplicação no perfil de produção	A aplicação utiliza a configuração do MySQL.

Observação: os cenários de cadastro, consulta, atualização e exclusão devem ser relacionados às operações efetivamente disponibilizadas pelo PersonController. A execução e aprovação de cada cenário deverão ser registradas após os testes.

2.4. Critérios de aprovação

Um teste será considerado aprovado quando o comportamento observado corresponder ao resultado esperado.

Também serão considerados:

- Ausência de erros inesperados durante a execução.
- Retorno adequado dos códigos HTTP.
- Persistência correta das informações.
- Tratamento adequado de exceções.
- Execução bem-sucedida da suíte automatizada.

2.5. Execução dos testes

A execução dos testes automatizados poderá ser realizada pelo terminal, na pasta principal do projeto, utilizando:

mvnw.cmd clean test

Após a execução, os relatórios poderão ser consultados na pasta target/site, incluindo os resultados do Maven Surefire e a cobertura do JaCoCo.

3. MATRIZ DE RASTREABILIDADE

3.1. Objetivo

A matriz de rastreabilidade relaciona as funcionalidades e requisitos do sistema com os testes planejados para verificar seu funcionamento.

Com ela, é possível identificar quais partes da aplicação precisam ser verificadas e acompanhar se todos os requisitos estão sendo contemplados durante os testes.

3.2. Matriz de rastreabilidade

ID	Requisito / funcionalidade	Componente relacionado	Teste relacionado
RF01	Disponibilizar comunicação por requisições REST	PersonController	CT01

RF02	Permitir cadastro de pessoas	PersonController / PersonService	CT02
RF03	Permitir consulta de pessoas	PersonController / PersonRepository	CT01 e CT03
RF04	Tratar consultas de registros inexistentes	PersonService / ControllerExceptionHandler	CT04
RF05	Permitir atualização de informações	PersonService / PersonRepository	CT05
RF06	Permitir exclusão de registros	PersonService / PersonRepository	CT06
RF07	Tratar entradas inválidas	PersonController / ControllerExceptionHandler	CT07
RF08	Tratar exceções da aplicação	ControllerExceptionHandler	CT08
RF09	Persistir informações no banco de dados	PersonRepository / JPA	CT09
RF10	Permitir utilização do banco H2 em testes	Configuração do perfil test	CT10
RF11	Permitir utilização do MySQL em produção	Configuração do perfil prod	CT11

3.3. Critério de rastreabilidade

Cada requisito identificado possui pelo menos um cenário de teste associado. Dessa forma, é possível acompanhar quais funcionalidades foram verificadas e identificar possíveis pendências.

A matriz também poderá ser atualizada caso novas funcionalidades sejam adicionadas ao projeto.

4. ANÁLISE DE RISCOS

4.1. Objetivo

A análise de riscos tem como finalidade identificar situações que podem prejudicar o funcionamento da aplicação ou dificultar sua manutenção e utilização.

Foram considerados os principais pontos de atenção relacionados à arquitetura, à comunicação entre as camadas e à utilização dos bancos de dados.

4.2. Identificação dos riscos

ID	Risco identificado	Probabilidade	Impacto
R01	Falha na conexão com o banco de dados MySQL	Média	Alto
R02	Erros durante a persistência das informações	Média	Alto
R03	Falhas no tratamento de exceções	Média	Médio
R04	Dados inválidos enviados nas requisições	Alta	Médio
R05	Diferenças de configuração entre H2 e MySQL	Média	Alto
R06	Falhas identificadas somente durante a integração das camadas	Média	Alto
R07	Ausência de cobertura de testes em determinadas funcionalidades	Média	Médio

4.3. Medidas preventivas e corretivas

R01 – Falha na conexão com o MySQL

Manter as configurações de conexão atualizadas, verificar se o serviço do MySQL está ativo e conferir usuário, senha e nome do banco de dados antes de executar a aplicação.

R02 – Erros na persistência dos dados

Realizar testes na camada PersonRepository e verificar se as operações de gravação, consulta, atualização e exclusão estão funcionando corretamente.

R03 – Falhas no tratamento de exceções

Verificar se o ControllerExceptionHandler está tratando adequadamente os erros e retornando respostas compreensíveis ao cliente.

R04 – Dados inválidos

Realizar testes com diferentes entradas, incluindo campos vazios, valores incorretos e informações fora do formato esperado.

R05 – Diferenças entre os bancos de dados

Executar os testes no perfil H2 e verificar a compatibilidade das configurações utilizadas no MySQL.

R06 – Problemas na integração das camadas

Realizar testes de integração para verificar a comunicação entre PersonController, PersonService e PersonRepository.

R07 – Cobertura insuficiente de testes

Utilizar o JaCoCo para acompanhar a cobertura dos testes e identificar classes ou métodos que ainda precisam ser verificados.

4.4. Classificação e acompanhamento

Os riscos identificados deverão ser acompanhados durante o desenvolvimento e a execução dos testes.

Os riscos classificados com impacto alto devem receber atenção prioritária, principalmente aqueles relacionados à conexão com o banco de dados e à persistência das informações.

A identificação antecipada desses problemas pode evitar falhas durante a utilização da aplicação e reduzir a necessidade de correções posteriores.

5. CONSIDERAÇÕES FINAIS

A documentação da arquitetura, o plano de testes, a matriz de rastreabilidade e a análise de riscos permitem compreender melhor a organização e o funcionamento do projeto. A separação da aplicação em camadas facilita a identificação de problemas e contribui para a manutenção do código. Além disso, a utilização de testes automatizados permite verificar o comportamento das funcionalidades e aumentar a confiabilidade do sistema.
Com esse planejamento, a equipe consegue acompanhar o desenvolvimento de maneira mais organizada, identificando pontos que precisam de atenção e verificando se as funcionalidades estão atendendo ao comportamento esperado.